

Материали за подготовка — База данни

Клас: 116 | Предмет: База данни

Учебен срок: втори срок, 2025/2026

1. Извличане на данни: SELECT, FROM, WHERE

- Роля на **SELECT** (кои колони влизат в резултата) и **FROM** (от коя таблица).
- **WHERE** ограничава редовете; сравнения с **=**, **>**, **>=** и др.
- Текстови литерали в единични кавички; числата — без кавички.
- **SELECT *** означава „всички колони на таблицата“ за този ред от резултата (удобно, когато изрично се иска целият запис преди ограничаването по **WHERE**).

Пример (конкретни колони)

```
SELECT code, title
FROM books
WHERE year_published >= 2020;
```

Обяснение: Заявката връща само избраните колони и само редовете, които удовлетворяват условието. Редът е фиксиран: първо **SELECT**, после **FROM**, после **WHERE** (ако има филтър).

Пример (всички колони)

```
SELECT *  
FROM loans  
WHERE member_id = 42;
```

Обяснение: ***** разгръща до всички полета по дефиницията на таблицата при изпълнение; при по-големи таблици в практика често се избират само нужните колони за по-бързо и по-ясно представяне на данните.

Пълни образци: равенство по текст и граница по число

Същата структура **SELECT ... FROM ... WHERE ...** с типични полета от учебни задачи (имената са само за ориентиране).

```
SELECT warehouse_code, warehouse_name  
FROM warehouses  
WHERE city = 'Пловдив';
```

```
SELECT course_code, credits  
FROM courses  
WHERE credits >= 6;
```

2. Външен ключ и каскади (FOREIGN KEY ... REFERENCES ...)

- Външният ключ държи колоната да сочи към валиден ред в родителската таблица.
- `REFERENCES родител(колона_ключ)` указва към какво се връзва FK колоната.
- `ON DELETE` / `ON UPDATE` с `CASCADE` (или други действия) задават поведението при промяна в родителя.
- Име на ограничение с `CONSTRAINT име` е по избор — помага за по-ясни дефиниции.

Пример

```
CONSTRAINT fk_loan_book
  FOREIGN KEY (book_id)
    REFERENCES books(id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
```

Обяснение: `FOREIGN KEY` и `REFERENCES` са задължителната двойка за деклариране на връзката. `CASCADE` каска промените според настройките на СУБД; важно е да различавате кое се отнася до изтриване и кое — до обновяване.

Пълни образци на декларации (различни по форма)

Често задачата дава само фрагмент от декларацията — тук са завършени два типични вида запис на външен ключ и каскади.

```
FOREIGN KEY (customer_id)
  REFERENCES customers(id)
  ON DELETE CASCADE
  ON UPDATE CASCADE
```

```
invoice_id BIGINT NOT NULL,  
CONSTRAINT fk_items_invoice  
    FOREIGN KEY (invoice_id)  
        REFERENCES invoices(invoice_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE
```

3. Псевдоними на колони с AS

- С **AS** дадено име в резултата не променя реалната колона в таблицата.
- Може да имате смес: част от колоните с **AS**, част без.
- **WHERE** се пише след изброяването на колоните и **FROM**, обикновено с истинското име на колоната в таблицата.

Пример

```
SELECT last_name AS фамилия,  
       salary AS заплата_евро  
FROM staff  
WHERE department_id = 2;
```

Обяснение: **SELECT** строи „екрания“ резултат; алтернативните имена улесняват четенето. Условието по **department_id** филтрира редовете преди да се покажат.

Пълни образци: смес от колони с и без **AS**

```
SELECT product_name AS име_на_продукт,  
       price AS цена_евро,  
       stock_quantity AS наличност  
FROM products  
WHERE category_id = 3;
```

```
SELECT first_name AS име,  
       last_name AS фамилия,  
       email AS електронна_поща  
FROM students  
WHERE city = 'Пловдив';
```

4. Подредба на резултат: ORDER BY

- **ORDER BY** колона **ASC** — възходящ ред (подразбира се, ако не пишете **ASC**).
- **ORDER BY** колона **DESC** — низходящ ред.
- Може да комбинирате избор на колони (**SELECT**) без **WHERE**, ако няма филтър по редове.

Пример

```
SELECT ticket_no, opened_at  
FROM support_tickets  
ORDER BY opened_at DESC;
```

Обяснение: **ORDER BY** не премахва редове; само определя подредбата на тези, които вече са избрани от **WHERE** (или всички, ако няма **WHERE**).

Пълни образци: **DESC** и **ASC**

```
SELECT employee_id, last_name, hire_date  
FROM employees  
ORDER BY hire_date DESC;
```

```
SELECT code, supplier_name, country  
FROM suppliers  
ORDER BY supplier_name ASC;
```

5. Филтриране по редове: едно или две условия

- Сравнение с число или дата под формата на низ в кавички.
- Свързване на две условия с **AND** — трябва да са изпълнени едновременно.

Пример

```
SELECT id, hall_name  
FROM cinemas  
WHERE city = 'София' AND screens >= 3;
```

Обяснение: Със **AND** и двете части на условието трябва да са верни за един и същ ред, за да попадне в резултата. Ако в по-сложни заявки смесвате **AND** и **OR**, използвайте скоби, за да няма двусмислие.

Пълни образци: едно условие, две с **AND**, и **AND** с **IS NULL**

```
SELECT title, pages  
FROM books  
WHERE pages > 400;
```

```
SELECT event_id, title, venue  
FROM events  
WHERE event_date = '2026-05-05' AND capacity >= 120;
```

```
SELECT *  
FROM rentals  
WHERE customer_id = 701 AND return_date IS NULL;
```

6. Липсващи стойности и шаблон (IS NULL, LIKE)

- Проверка „няма стойност“ се пише `колона IS NULL` (не `= NULL`).
- `LIKE 'текст%'` — имена или низове, които **започват** с дадения префикс; `%` е „всеки край на низа“ според SQL.
- Комбинирайте изискванията с `AND`, когато трябва да са верни заедно.

Пример

```
SELECT asset_tag, note
FROM equipment
WHERE disposed_at IS NULL
      AND note LIKE 'Експеримент%';
```

Обяснение: `IS NULL` е специализиран синтаксис за NULL. Двата реда на условието с `AND` ограничават множеството записи конюнктивно.

Пълен образец: `IS NULL` и префикс с `LIKE`

```
SELECT sku, product_name
FROM devices
WHERE barcode IS NULL
      AND product_name LIKE 'Мобилен%';
```

7. Съединяване на таблици: INNER JOIN ... ON ...

- Вътрешно съединение връща само редове с намерени съвпадения и в двете таблици.
- **ON** задава условието за съчетаване (обичайно FK = PK на родителя).
- Псевдоними с **AS** за таблица (**t AS x**) съкращават текст в **SELECT** и **ON**.

Пример

```
SELECT p.title, u.login
FROM posts AS p
INNER JOIN users AS u ON p.author_user_id = u.id
WHERE p.published = true;
```

Обяснение: Свързващите колони в двете таблици трябва да съответстват по тип и смисъл. В учебни задачи често се дописва само фрагмент от синтаксиса за съединението — важно е да помните къде стои **INNER JOIN**, къде **ON** и че условието за съчетаване обикновено използва **=** между ключовите колони.

Пълни образци: съединение с псевдоними на таблици

```
SELECT c.last_name, o.total_amount, o.order_date
FROM orders AS o
INNER JOIN customers AS c ON o.customer_id = c.id
WHERE o.total_amount > 250
ORDER BY o.order_date DESC;
```

```
SELECT i.invoice_id, c.last_name, i.total_amount
FROM invoices AS i
```

```
INNER JOIN clients AS c ON i.client_id = c.client_id
WHERE i.invoice_date >= '2026-01-01'
ORDER BY i.total_amount DESC;
```

8. INSERT: съответствие колона ↔ стойност

- След `INSERT INTO име (col1, col2, ...)` реда в `VALUES (...)` са в същия ред като имената на колоните.
- Промяната само на подредба в списъка с колони изисква съща подредба в `VALUES`; не смесвайте с „логическата“ последователност в текста над заявката.
- Първичен ключ с авто номер може да липсва в списъка с колони — тогава се генерира от системата.

Пример

```
INSERT INTO members (joined_on, role, username)
VALUES ('2026-03-01', 'reader', 'n.petrova');
```

Обяснение: Първата стойност отива в `joined_on`, втората в `role`, третата в `username`. Ако разместите имената след `INTO`, трябва да преместите и стойностите в същия ред.

Пълни образци: подредба на литерали според списъка с колони

Редът в `VALUES` следва реда на имената след `INTO` — не „логическия“ ред от поясненията към задачата.

```
INSERT INTO customers (email, last_name, first_name, city)
VALUES ('elena.g@example.org', 'Георгиева', 'Елена', 'София');
```

```
INSERT INTO products (price, category_id, product_name, stock_quantity)
VALUES (399.99, 4, 'Монитор', 12);
```

9. Първичен и външен ключ върху схема

- **PK** — уникален ключ на реда; **FK** — колона към PK в родител; на чертежа: родител → дете по веригата от данните.

customers → orders → order_items

FK: orders.customer_id → customers.id | FK: order_items.order_id → orders.id

customers

id · PK
first_name
last_name
email
city

orders

id · PK
customer_id · FK→customers.id
order_date
total_amount

order_items

id · PK
order_id · FK→orders.id
product_name
quantity
unit_price

clients → invoices → invoice_items

FK: invoices.client_id → clients.client_id | FK: invoice_items.invoice_id → invoices.invoice_id

clients

client_id · PK
first_name

invoices

invoice_id · PK
client_id · FK→clients.client_id

invoice_items

item_id · PK
invoice_id · FK→invoices.invoice_id

last_name
email

invoice_date
total_amount

product_name
quantity
unit_price

Приятно учене!

